

api.rental-fleet.com - Étape 1 - Configurations, Erreurs, Modèles et BD

Objectifs

- vous familiariser avec la structure du projet
- voir comment fonctionne l'export/import de modules en JS
- mettre en place des configurations et erreurs personnalisées
- apprendre à séparer le rôle de chaque composant de MVC et du projet
- installer une librairie par NPM et apprendre à utiliser MariaDB en Node.js
- créer un modèle pour accéder aux données de la BD
- utiliser le modèle dans le contrôleur
- ajuster la vue et tester et analyser l'API avec HTTPie

Avant de débiter, tester l'API dans son état actuel avec `/products` , `/products/1` et `/products/99`

1- Installation de la librairie MariaDB

- Voir le document **NPM - Node Package Manager.pdf**

2- Créez la base de données

- Utilisez le script **create-api-rental-fleet-com.sql**

3- Créer le fichier .env

- Copier **.env.example** et renommer la copie en **.env**
- Ajustez les configurations pour la base de donnée

Les fichiers **.env** servent à **définir des données reliées au contexte** dans lequel le code s'exécute.

Sur un environnement de **développement**, de **test**, de **production**, etc. **Chacun aura son fichier .env**

4- Dossier de fichiers de configurations, constantes et d'énumérations

- Créer le dossier **configs** sous **src**
- Ajouter à **configs** le fichier **env.js** qui donnera accès aux informations de **.env** sous la forme d'un objet
- Créer dans **configs** un fichier **index.js** qui contiendra la ligne **export * from './env.js'**;
- Ajouter une entrée dans la section **imports** du fichier **package.json** pour créer un alias vers le fichier configs/index.js :

```
"#configs": "./src/configs/index.js",
```

La technique du "Barrel Pattern" (fichier Barrel, chargement par lot)

Consiste à regrouper et réexporter plusieurs modules ou fichiers au sein d'un fichier point d'entrée unique (généralement nommé index.js).

L'idée ici est d'avoir un seul chemin absolu **#configs** qui offrira toutes les configurations, nous n'aurons qu'à sélectionner celles qu'on veut.

Pour le moment nous n'avons accès qu'à **DB_CONFIG**.

Remarquez que pour cet alias, nous ne terminons pas le chemin par **/*** mais bien **/index.js** car **on veut pointer sur un fichier** spécifique.

La création d'un lot de configuration est réellement pour la **simplicité de l'utilisation** que nous en ferons plus tard.

On peut se permettre d'importer **#configs** dans d'autres modules car c'est du **code très léger** car ne contient que des constantes.

5- Gestion des erreurs de Base de données

- Ajoutez le fichier **db-error-codes.js** au dossier **configs** et faites en sorte qu'il soit accessible dans le chargement par lot **#configs**, comme ce qu'on a fait pour **env.js**
- Ajouter le fichier **error-types.js** au dossier **utils**
- Ajouter le fichier **handle-db-errors.js** au dossier **utils** et faites les imports nécessaires pour avoir accès à **DB_ERROR_CODES** et **ErrorTypes** dans ce fichier

6- Création de l'infrastructure base de données

- Créer le dossier **infrastructure** sous **src**
- Ajoutez y le fichier **db.js** (l'objet **pool** va gérer l'accès à MariaDB)
- * N'oubliez pas de créer un alias pour le chemin vers le dossier **infrastructure**

7- Création du Modèle représentant les données des produits

- Créer le dossier **models** sous **src**
- Ajouter le fichier **product-model.js** et faites les imports nécessaires pour avoir accès au **pool** de l'infrastructure de BD ainsi qu'à la fonction **handleDBError**
- * N'oubliez pas de créer un alias pour le chemin vers le dossier **models**

Le modèle gère les erreurs de MariaDB et lance une Erreur attrapée par le contrôleur. **Séparation des rôles.**

8- Utilisation du modèle par le contrôleur

- Remplacer le fichier **product-controller.js** et ajoutez les imports pour le modèle de produit et **ErrorTypes**

Remarquez la gestion d'erreur lancée par le modèle, on analyse le type de l'erreur et on ajuste la réponse.

Comme **un appel à la base de données peut être long et bloquer le fil principal** nous utiliserons **await** devant la méthode **selectAll** et **selectById**. **await** peut être utilisé seulement dans une fonction **async**.

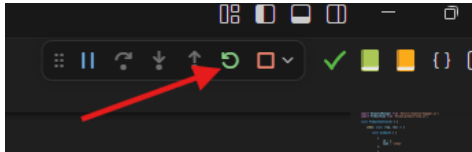
Notez que pour **const id = res.locals.url.params.id;** on ne valide actuellement pas le type de donnée.

9- Ajuster la vue de produits

- Ajouter les attributs manquants afin d'avoir les 4 champs dans le résultat

10- Tester grâce à HTTPie

- Créer une collection **api.rental-fleet.com** et ajoutez les routes :
GET /products , **GET /products/1** , **GET /products/99**
- **N'oubliez pas de recharger le processus de l'API** pour que les modifications soient prises en compte.



Request GET Response 200

▶ HTTP/1.1 200 OK (6 headers)

```
1 ▼ [
2 ▼ {
3   "id": 1,
4   "name": "Hand Cart",
5   "picture": "hand-cart.avif",
6   "price": 118
7 } ,
8 ▼ {
9   "id": 2,
10  "name": "Moving Blanket",
11  "picture": "moving-blanket.avif",
12  "price": 24
13 } ,
14 ▼ {
15  "id": 3,
16  "name": "Packaging Tape",
17  "picture": "packaging-tape.avif",
18  "price": 19
19 }
20 ]
```

Request GET Response 200

▶ HTTP/1.1 200 OK (6 headers)

```
1 ▼ {
2   "id": 1,
3   "name": "Hand Cart",
4   "picture": "hand-cart.avif",
5   "price": 118
6 }
```

Request GET Response 404

▶ HTTP/1.1 404 Not Found (5 headers)

```
1 Ressource does not exists
```

À ce point, votre projet devrait avoir la structure suivante.
On peut voir les ajouts en vert et les modifications en jaune.

```
▼ 420-kbg-rental-fleet-api
  > .vscode
  > node_modules
  ▼ src
    ▼ configs
      JS db-error-codes.js
      JS env.js
      JS index.js
    ▼ controllers
      JS product-controller.js
    ▼ infrastructure
      JS db.js
    ▼ middlewares
      JS match-route.js
      JS parse-url.js
      JS set-default-headers.js
    ▼ models
      JS product-model.js
    ▼ routes
      JS product-routes.js
    ▼ utils
      JS error-types.js
      JS handle-db-errors.js
      JS response-magager.js
      JS run-middlewares.js
    ▼ views
      JS product-view.js
  JS index.js
  ⚙ .env
  ⚙ .env.example
  💎 .gitignore
  📄 LICENSE
  {} package-lock.json
  {} package.json
  ⓘ README.md
```